Governed Setpoint Writing for Industrial Agents: A Configuration-Driven Runtime and Simulated Evaluation

***Abstract*—Industrial agents can propose process changes, but a device write must be resolved against the active production context, control point, and current limits. This paper presents a configuration-driven runtime that binds line, product, recipe, and run context to direction-specific signal mappings. Task-mediated and direct calls share dispatch; route handlers resolve live state, perform applicable revalidation and approval, and pass writes through common guards. This separation makes a task's initial context descriptive rather than a grant of authority. Command outcomes and optional device readbacks are recorded separately from subsequent process observations. Across four simulated scenes, 220/220 admissible mapping probes were accepted, 219/219 collected structural-range probes were rejected, and 216/216 available simulated readbacks matched. In a mock-driver ablation, all six targeted violations returned HTTP 400 with the process-window guard enabled and HTTP 200 with it removed. In a separate three-scene comparison, 15/15 scripted and 0/15 no-write endpoints were in band, but their different endpoints and horizons preclude an efficacy estimate. The live-Agent pilot issued no governed writes. The evidence supports selected simulator and mock-driver paths, not process-control efficacy or physical-plant safety.**

***Index Terms*—Industrial agents, governed setpoint writing, industrial communication, simulation-based evaluation, system integration.**

I. Introduction

INDUSTRIAL agents can propose process changes, but a proposal becomes an industrial operation only after the runtime resolves the active production context and maps the request to the intended control signal. This mapping is scene dependent. For example, a cast-film line-speed setpoint may affect measured film thickness, but the setpoint and measurement are distinct process variables even when both use MQTT. Command acceptance, device readback, and a later process measurement also provide different evidence: acceptance records the service outcome, readback reports the actuator value, and the process measurement records what followed. Fig. 1 illustrates this distinction.

Industrial-agent research has addressed planning, control generation, PLC programming, and fault recovery. Other work exposes industrial capabilities through protocol adapters or semantic commissioning, while plant-facing systems apply approval and verification mechanisms. These studies address different tasks and report different evidence, so their results are not directly comparable with those of the runtime studied here. This paper examines how these concerns meet at the boundary between a task-level proposal and a device operation. At that boundary, the runtime must preserve production context and signal meaning across task-mediated and direct calls, recheck applicable authority and limits at execution, and distinguish command evidence from subsequent process observations. Agent planning, protocol access, approval, and range checks are not individually new contributions of this work.

We implement this boundary in a configuration-driven runtime. A scene binds the line, product, recipe, and run to direction-specific acquisition and control nodes, registered protocol drivers, process objectives, and write limits. Task-mediated and direct calls enter a shared dispatcher; route handlers resolve live state and apply the checks required for that route before driver dispatch. For example, when approval is required, a recipe operation rechecks its active production context after approval, whereas a single-node operation rechecks its binding and any applicable grant and applies current limits without using the same context snapshot. These checks address different stale-state risks; they do not make validation and device I/O atomic. The contribution is the integration of process meaning, route-specific authority, and execution evidence, rather than a new control law or agent-reasoning algorithm.

The contributions are:

1) A production-scene configuration model that links operational context and direction-specific signal mappings to registered drivers, objectives, and limits for use at the operation boundary. (Section III.)
2) A task-to-device path for registered Agent harnesses and direct callers that combines shared dispatch, route-specific revalidation and conditional approval, and common write guards. It keeps command outcomes, optional readbacks, and later process observations distinct. (Sections III–V.)
3) An evidence-bounded evaluation of configured mappings and pre-dispatch guards across four simulated scenes, with command readback, process observation, and live-Agent outcomes reported as separate evidence. (Section VI.)

Section II reviews industrial-agent, interface, and operation-governance work. Sections III–V describe the runtime configuration, operation path, and write controls. Section VI reports the evaluation by evidence type, and Section VII discusses the limits of the results.

II. Related Work

Table I summarizes representative systems discussed below. They address different tasks, interfaces, and evaluation units, so the comparison is qualitative rather than a ranking or head-to-head benchmark. A more detailed source list is provided in Table S7-I.

TABLE I
REPRESENTATIVE RELATED WORK AND THE SCOPE OF ITS REPORTED EVALUATION. THE STUDIES ADDRESS DIFFERENT TASKS AND ARE NOT A COMMON BENCHMARK.

Study	Main focus	Reported setting
Biskupovic et al. [1]	Agent-based industrial service selection and reselection	Service adaptability and performance in an industrial cyber–physical system
Köhle et al. [2]	Module-specific agents and skill-based manufacturing	Nine simulated production challenges
Graul et al. [3]	Grounded proposals, documented authorization, and PLC validation	Two laboratory proof-of-concept settings
Schall [4]	Process-diagram, SCADA, and procedure-grounded verification	Five simulated scenarios and proposal evaluation
Vyas et al. [5]	Multi-agent fault-recovery planning with a process-plant twin	Mixing and stirred-tank reactor studies
This work	Scene-bound signal mappings, shared dispatch, and route-specific governance	Simulator and mock-driver probes, scripted missions, and a negative live-Agent pilot

A. Industrial Agents and Plant Integration

Industrial-agent studies consider service supervision, control generation, PLC programming, and production tasks. Biskupovic et al. study online service selection and reselection in industrial cyber–physical systems. Xia et al. present a digital-twin-connected automation architecture and control generation; related work generates PLC programs or incorporates human instructions into model-predictive control [1], [6]–[9]. In recent TII work, Li et al. use structured instructions, retrieval, and iterative verification to generate and check finite-element scripts for Abaqus; that study evaluates engineering-artifact generation rather than runtime authority over process writes [10]. Other studies examine language-model agents, manufacturing skills, and natural-language machine interaction [2], [11]–[14]. Vyas et al. combine multi-agent fault-recovery planning and plant-knowledge retrieval with Digital Process Plant Twin studies of mixing and continuous stirred-tank reactors (CSTRs) [5].

These papers address agent organization, generated actions, or task execution. The present study does not introduce an agent policy or control law. It examines the runtime boundary that carries production context and direction-specific signal mappings into an operation and applies the applicable checks before dispatch.

B. Industrial Interfaces and Semantic Commissioning

Agent-facing interface work includes Model Context Protocol (MCP) adapters for industrial protocols, semantic recipes built from OPC UA skills, and tools for binding manufacturing data to a cognitive digital twin [15]–[18]. These systems address capability discovery, protocol access, or commissioning. Here, configuration binds known protocol drivers to scene-specific read and write nodes; a named operation resolves those bindings within the same runtime path that checks authority and write limits. We do not treat MCP or a semantic node name as an authorization mechanism.

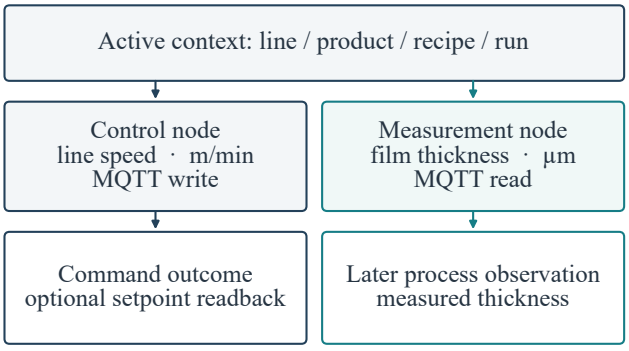


Fig. 1. Illustrative configured cast-film example: the line-speed setpoint and film-thickness measurement are separate nodes even though both use MQTT. Command acceptance or optional setpoint readback is distinct from the later process observation.

C. Governed Operations and Verification

Prior work already provides substantial plant-facing verification. Graul et al. ground proposals in the Asset Administration Shell and combine documented human authorization and PLC-validated OPC UA Methods [3]. Schall verifies proposals against piping and instrumentation diagrams, SCADA data, and standard operating procedures; Lin and Chen describe a policy-governed sensor–MES–ERP digital thread [4], [19]. PLCBench evaluates autonomous agents against commercial PLCs with hardware-in-the-loop process simulation [20]. These studies complement the runtime examined here and demonstrate that authorization and safety controls are not new. Our implementation instead examines how configured signal bindings, route-specific checks, shared dispatch, and separate command and process records operate together at the write boundary. This is a system-integration contribution; the studies use different settings and do not establish comparative safety.

D. Production Context and Outcome Evidence

MCP standardizes tool discovery and invocation; the application remains responsible for operation-specific authority and process constraints [21]. ISA-88 and ISA-95 define recipe and production-operation concepts [22], [23]. Run-to-run control relates process observations to subsequent decisions, while human–automation research examines allocation of responsibilities between people and automation [24], [25]. These works inform the production context and evidence distinctions used in this paper. The implementation does not claim conformance to a complete ISA-88/95 information model, and its scripted approval fixture is not a human-factors evaluation.

III. SYSTEM DESIGN

The runtime exposes named services through two entry paths. Task-mediated calls pass through the collaboration runtime and a registered Agent harness. Direct calls through the

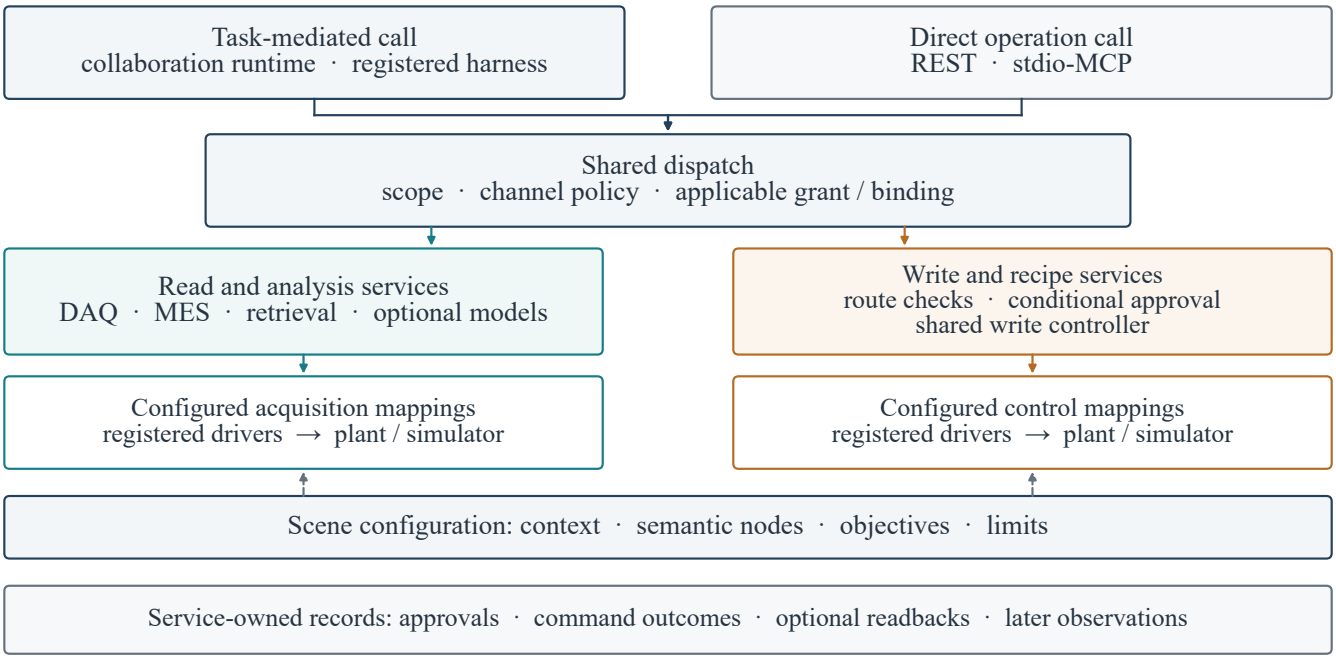


Fig. 2. The system routes task-mediated and direct operation calls through shared dispatch and service handlers. The service boxes show functional groupings rather than exclusive driver assignments: a configured MES-REST adapter can also implement a write mapping. Solid arrows show operation flow and configured field mappings; dashed connectors associate the scene configuration with the field mappings it defines; the read/analysis group is a functional grouping, not a claim that every service uses a field driver. Named write operations use the shared write controller, while service-owned records keep approvals, command receipts, optional readbacks, and later observations distinct. The driver box abstracts protocol adapters; the evaluated scene mappings also included HTTP.

Representational State Transfer (REST) API or the standard input/output (stdio) transport for the Model Context Protocol (MCP) enter the shared dispatcher without creating a task turn. Both paths use the scene configuration to resolve production context, semantic-node mappings, protocol endpoints, objectives, and write limits where applicable.

A. Design Rationale and Requirements

The runtime must preserve process meaning across heterogeneous drivers, provide a stable operation contract for registered harness adapters, and check current authority and process limits before each write. It must also distinguish command acceptance from later process observations. Scene configuration, registered adapters, shared dispatch, route handlers, and service-owned records enforce these boundaries.

At task start, the context builder assembles the active production tuple and relevant process summaries. This context informs a proposal but does not grant write authority; operation handlers resolve the live state needed by their routes at invocation.

B. Layered Architecture

Fig. 2 shows the system layers and their interfaces. Task and chat routes use a collaboration runtime with task coordination and registered harness adapters. An adapter presents the agent's scoped tools and task context; its binding and channel policy constrain the available operations.

Direct REST calls and the stdio-MCP operation bridge bypass task execution and enter the shared dispatcher. They create neither a task nor a harness turn; MCP management and task methods remain on their service routes. The scene configuration has two consumers: the task route receives active context and objectives, while operation services resolve node mappings and guards when an operation executes. The service band groups read and analysis services (DAQ, MES data access, retrieval, and optional model services) separately from write and recipe services. The driver registry can also select a configured MES-REST adapter for a write mapping. At the field layer, input/output (I/O) mappings connect semantic nodes to built-in or plugin drivers for a plant or simulator. Each owning service retains its task, command, observation, or configuration records and associates them with the actor and production context where supported.

C. Scene Configuration as the Integration Boundary

Scene-specific process semantics are configured before an agent task begins. The active line identifies the production context; product and recipe records supply applicable targets and limits; the run record provides the current operating state. Semantic nodes bind process variables to direction-specific acquisition or control mappings, and the driver registry resolves each mapping to a supported protocol implementation. Here *semantic* denotes a named process variable with a direction, unit, and configured mapping; it is not an OPC UA or Asset Administration Shell information model. Each agent-facing operation is named and typed, so a harness does not need to construct protocol-specific messages. A scene using an already registered driver can supply different context and node mappings behind the same operation contract. A protocol not present in the registry still requires a driver adapter; configuration alone does not implement a new protocol.

TABLE II
SCENE CONFIGURATION ELEMENTS AND THEIR ROLES IN THE RUNTIME PATH.

Element	Configured information	Runtime use
Production context	Line, product, recipe, and run associations	Context construction for a task turn; live context resolution before a write
Semantic I/O	Process variable, direction, unit, and node mapping	Acquisition uses the read mapping; accepted operations resolve the configured control mapping
Driver binding	Supported protocol driver and endpoint configuration	Driver registry selects the protocol implementation after operation checks
Operation admission	Engineering, baseline, product, and recipe limits; step and timing rules where configured	Route handler checks the candidate against current bounds and rejects inadmissible requests
Approval policy	Whether approval is required and the decision validity window	Pending request binds the proposed action; route-specific checks run after approval
Evidence ownership	Command receipt, verification grade, and later DAQ observations	Owning services retain command outcomes separately from process measurements

In the cast-film simulator preset, for example, the MQTT command mapping `linespeed-sp` controls line speed in m/min, while the distinct `film-thickness` mapping supplies thickness in μm. A task can refer to both nodes: a candidate is admitted against the control node's limits, whereas a later DAQ observation describes thickness. Sharing MQTT does not make either signal a substitute for the other.

Table II summarizes how the execution path uses the scene description. It distinguishes configuration-time bindings from checks against live state. The context assembled for an agent turn is descriptive: before a write, the handler resolves the active context again and applies the current limits. Approval is bound to the proposal and to the context where that route supports context snapshots; the command receipt remains separate from subsequent DAQ observations.

D. Cross-Module Execution Flow

For a task-mediated call, the registered adapter presents a scoped tool catalog and current task context to the Agent. At the operation boundary, the shared dispatcher checks the predicates declared for the named operation before selecting its handler. The handler resolves live production state, applies route-specific guards and limits, and requests any required approval before device I/O.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured.

After these predicates pass, the selected route applies its operational guards and any required approval. For example, an exploration request checks the control binding and DAQ visibility before entering the governed write path; an invisible target returns an error before a write. The checks performed after approval depend on the route; Section V specifies the single-node and recipe-operation paths.

For a direct REST call, the service establishes a and c before applying these checks. An agent token must match the requested agent identifier; an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy, followed by any applicable grant and binding checks. Handlers resolve live production state from request identifiers and owning services.

Section V specifies route-level admission, timing, approval, and evidence rules for both task-mediated and direct calls.

IV. AGENT-COORDINATED TASK-LEVEL OPERATION

A task supplies a process objective and the active production context, including line, product, and run. Fig. 3 shows the conceptual task-to-write workflow in the current design, not a successful live-Agent run from this evaluation. The task context alone does not authorize a device write. The control node is resolved from an explicit `control_node_id` or the Agent's DCW/recipe binding. The data-acquisition (DAQ) target is resolved from an explicit target ID or the Agent's DAQ binding; a read-only node visible through the channel line is also eligible. Section VI reports the scripted process results separately from the negative live-Agent pilot.

In goal-mode execution, a Lead Agent may decompose the objective, dispatch subtasks to worker agents, and judge completion. This optional coordination organizes the task but does not define a control policy. For each control step, the responsible Agent invokes `optimization_explore`, which performs the checks before device I/O.

The operation resolves the optimization profile and control-node binding, then uses the node's stored value as the current setpoint u_0. This value is not necessarily a fresh device readback. If direction is omitted, the operation defaults to an upward adjustment; otherwise, the supplied direction determines $s \in \{-1, +1\}$. For a step Δu, it forms

$$u^* = \mathrm{clip}(u_0 + s\Delta u,\, u_{\min},\, u_{\max}), \tag{2}$$

where configured tuning bounds take precedence; node bounds are used otherwise. The effective step is selected in this order:

an explicit argument, the binding's tuning step, the scene's maximum step, or one twentieth of the node range.

The operation then resolves the DAQ target and verifies its visibility. In the later source implementation (commits `1aa4b136` and `66a8d5b5`), an optional identifiability preflight checks the control node's intervention ledger: an unclosed prior intervention or an unexpired recommended window after an underpowered or confounded verdict blocks a new step. A model–ontology cross-check treats missing evidence as unknown; a confirmed conflict rejects an automatic request; on the manual path, this cross-check does not itself block dispatch. Either gate can be disabled by configuration. If the checks pass, the operation attempts a pre-write DAQ read; its mean may be unavailable. These later checks are design features, not results attributed to the November 1 experiments, whose manifests name `da627d21` and whose active services were not captured as a hermetic snapshot.

A valid candidate enters the shared write-control path, which resolves the applicable binding and authority and determines whether the route requires approval. Only a gated route waits for approval. After approval, the path rechecks authorization and binding; the common controller applies the effective range and, for agent or manual writes outside benchmark mode, the applicable step, timing, and hold constraints before calling the configured driver. A route without an approval gate skips the wait but still passes through the common controller checks. A write denial or approval decline is recorded as rejected without driver dispatch. A driver-stage error may also be recorded as rejected after dispatch has begun, so status alone cannot establish whether the device changed; the caller must read and reconcile device state before retrying.

After a successful write, the operation waits for the configured settling interval (10–300 s; 45 s by default) and attempts a post-write DAQ read. In the later implementation, let t_0 be the timestamp before the pre-write read and governed write call. The pre-write window is $[t_0 - \tau, t_0]$ and the post-write window is $[t_0, t_1]$ after waiting τ. Approval may delay dispatch, so the latter can include pre-dispatch samples and its mean alone is not a causal estimate. The earlier `da627d21` source instead reads trailing 45-s windows; neither window definition establishes causation.

The exploration record retains the control value (the proposed target after a successful write), step, available DAQ means, and write status/message. For a rejected attempt, it retains the status/message but no separate attempted-target field. The shared write path separately records the command outcome and any device readback. On a later task turn, the Agent can review the returned result and choose another direction and step or stop. Thus, iteration depends on a subsequent Agent decision; the runtime does not autonomously iterate to convergence. Fig. 3 summarizes the operation.

V. Operation Governance and Evidence

Section III defines the shared dispatch contract. This section focuses on the checks that follow it: scene-specific write admission, conditional approval, and the records returned by each route.

A. Scene-Specific Write Admission

After the shared dispatch checks pass, the selected handler applies the limits configured for the target scene and control node. Authorization and value admission are separate checks. The target band describes a desired process outcome; it does not define the admissible write interval. A candidate may satisfy all write constraints while the measured process value remains outside the target band.

For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \qquad (3)$$

where $\mathcal{H}_n$ is the hard engineering range, and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits. These numerical bounds are distinct from the scope, policy, grant, and binding predicates in Eq. (1). Missing optional limits impose no restriction. An empty intersection rejects the request and identifies the excluding layer. Recipe dispatch checks a replacement recipe against the hard, baseline, and product limits rather than the previous recipe's window. Trial overrides are additionally checked against the applicable recipe window. Outside benchmark mode, step, interval, and hold-window rules apply to writes whose source is agent or manual; recipe and rollback sources follow their own paths. These application-level checks complement, but do not replace, equipment interlocks.

B. Approval, Execution, and Evidence

Approval behavior depends on the operation route. For a single-node `optimization_explore` request in manual mode, the handler sends the node, requested value, current effective interval, and stated rationale to the approval service, then waits before calling the governed write operation. A decline or timeout returns without reaching the controller write. While the request is pending, the handler does not dispatch. After approval, it rechecks the applicable node binding and channel grant. The shared write controller then evaluates the current engineering, parameter, product, and recipe limits; outside benchmark mode, agent or manual writes also face step, interval, and hold-window checks before dispatch to the configured driver. This single-node path does not use the recipe-operation context snapshot described below. Revalidation does not by itself establish an atomic snapshot spanning the external device write.

Recipe operations use a separate approval path. For `recipe_trial` and `recipe_apply`, the approval is bound to the recipe operation and active line–product–recipe–run context. After approval, the handler checks the authority and context again and verifies the recipe definition and candidate scope before dispatch. A changed context, revoked binding, or changed recipe definition prevents that proposal from proceeding. Recipe trial approvals use a 30-minute window by default; single-node approval waits expire after 180 s by default. Outside benchmark mode, agent and manual writes are subject to a minimum 60-s interval following a setpoint change; recipe trials also use a per-line cadence of 300 s by default.

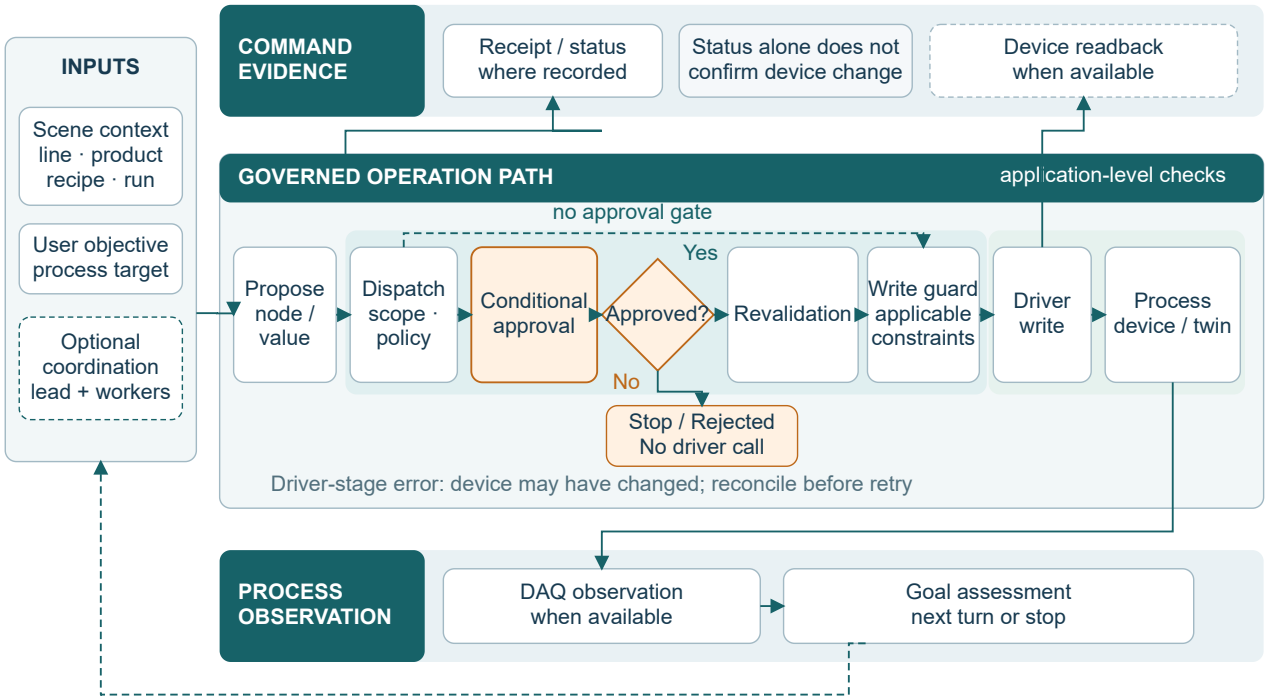


Fig. 3. Conceptual task-to-write flow. Approval is route-dependent: gated routes revalidate after approval; other routes bypass the gate and enter the shared write guard. A pre-dispatch rejection makes no driver call, whereas a driver-stage error may leave device state changed and requires reconciliation. Receipt, optional device readback, and later DAQ observation are distinct evidence. The figure describes the design, not an observed live-Agent success.

Fig. 4 contrasts the bounds used for online node writes in Eq. (3) with those used for a replacement recipe, alongside the route-dependent approval and shared-dispatch paths.

A `recipe_trial` illustrates the recipe-operation path. The handler checks the bound recipe, active run, candidate parameters, and route timing. If approval is required, it requests approval and then revalidates authority and context before submitting a bounded batch through the configured driver. It returns a result for each parameter. In the current implementation, parameters are attempted individually, so a batch can be partially applied; the per-parameter results and device state must be reconciled before retrying. A trial does not save or apply a recipe version; the caller can retrieve later DAQ observations separately before updating, applying, or rolling back the recipe. The driver registry maps semantic controls to configured protocol drivers, so operation arguments do not contain protocol-specific details.

Receipts record the command outcome; observations record what followed in the process. Command results are graded as readback-verified, transport-acknowledged, or unverified. A readback mismatch can trigger up to three reads at 2 s intervals, and recipe batches retain verification outcomes for each parameter. A readback-verified result means the returned value matched the request. A transport acknowledgement alone does not establish that the device applied the value, and an unverified result does not establish a matching readback. DAQ stores later timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section VI evaluates selected parts of this evidence chain under the stated test conditions.

VI. Experimental Evaluation

The frozen core blocks ran against already-running platform and simulator services; most used benchmark mode, which relaxes normal timing and hold-window controls. Separate production-mode guard checks are reported on their own. No cold-start deployment or physical-plant run was performed. Results are presented in the order an operation crosses the runtime: configured mapping, authorization and write admission, process observations, and Agent-level execution. The blocks use different units—mapping requests, fixed stimuli, per-seed endpoints, a single simulated-twin execution, and Agent episodes—so counts are not pooled.

Table III connects each evaluation block to the question it can answer. A matching readback checks the actuator value returned by the simulator; it does not establish a process response. Conversely, an in-band process observation does not by itself establish which write caused it. The route and guard checks address pre-dispatch behavior, whereas the live-Agent episodes address whether a task reached a governed write. These distinctions determine the scope of the conclusions below.

A. Configured Mapping and Integration Checks

Two integrated executions each passed 77 assertions across 17 phases. These repeat checks of the configured service path; the assertions are not independent trials.

The two executions used seed 42, started with the platform and simulator already running, and did not activate an Agent harness. A separate preset check within the integrated film-line

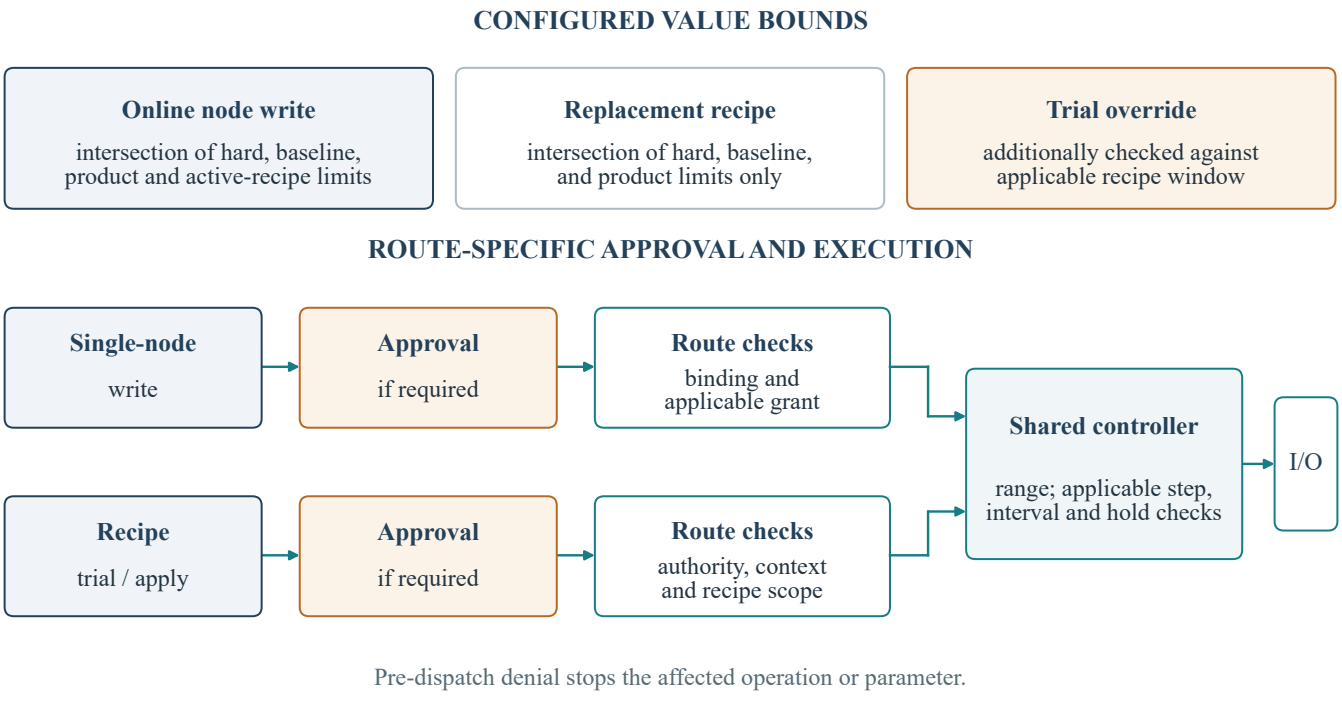


Fig. 4. Write admission and route-dependent approval. An online node write uses the intersection of hard, baseline, product, and active-recipe bounds; a replacement recipe is checked against hard, baseline, and product bounds, not the previous recipe window. Trial overrides are additionally checked against the applicable recipe window. After required approval, the routes recheck their applicable binding, grant, context, and recipe scope. The shared controller applies range checks; step, interval, and hold-window checks apply to agent or manual sources outside benchmark mode. A failed pre-dispatch check prevents the affected operation or parameter from reaching the driver.

TABLE III
EVALUATION BLOCKS, OBSERVATIONAL UNITS, AND LIMITS OF INTERPRETATION.

Block	Unit	Observation	Interpretation boundary
Mapping probes	Configured request	Acceptance, rejection, and available simulated readback	Same configured limits supply inputs and checks; no physical actuation
Guard ablation	Fixed stimulus and technical repetition	HTTP response status	One scene; benchmark mode; no readback-mismatch stimulus
Scripted missions	Seed-level endpoint	Process value relative to its target band	Unmatched horizons and endpoint rules; no controller baseline
Injection-twin pilot	One scripted execution	Write receipt, readback, and later process observation	No replication; incomplete raw series and version provenance
Live-Agent pilot	Recorded episode	Governed writes and reported task status	Two episodes; neither issued a governed write

path configured five endpoint types and rejected nine selected negative probes. It reused the same runtime, so it is not a cold-start deployment or an independent replication of the mapping matrix.

The scene–protocol probes covered 55 writable nodes across four simulated scenes and five configured protocol variants. Each node received three interior-value requests, one upper-boundary request, and four structural-range requests. All 220 admissible requests were accepted; 219 collected structural-range requests were rejected; and all 216 available readbacks matched. These probes check the consistency of configured mappings: the tested values and guards use the same configured limits. Simulator readback echoes the configured value and does not verify physical actuation. One structural request was not collected, HTTP nodes do not expose readback, and the full scene–protocol matrix remains incomplete. The excluded cast-film run and other deviations are documented in Supplementary File S7.

B. Write-Guard and Route Checks

The ablation used one scene, direct API calls, a mock driver, and three deterministic repetitions per guard setting in benchmark mode. Each repetition submitted four structural-range violations and two process-window violations, as well as three legal and three boundary requests. The timed writes used for latency summaries were excluded from these counts. With all guards enabled, all six targeted process-window violations returned HTTP 400. With the process-window check removed, all six returned HTTP 200; structural violations continued to be rejected. The archived records contain neither response bodies nor independent per-request driver-call logs, so HTTP 200 alone does not establish driver invocation. The four structural violations were rejected in every repetition and setting; the nine legal requests per setting were accepted

TABLE IV
COLLECTED SCENE–PROTOCOL MAPPING RESULTS. READBACK COUNTS INCLUDE ACCEPTED REQUESTS FOR WHICH THE SIMULATOR EXPOSES AN ACTUATOR VALUE.

Scene	Writable nodes	Protocols represented	Admissible accepted	Structural rejected	Readback matched
Injection	11	TCP 5; RTU 2; OPC UA 3; MQTT 1	44/44	44/44	44/44
Wastewater	7	TCP 1; RTU 3; OPC UA 2; MQTT 1	28/28	28/28	28/28
Annealing	7	TCP 3; RTU 2; OPC UA 1; MQTT 1	28/28	28/28	28/28
Biaxial line	30	TCP 20; RTU 1; OPC UA 8; HTTP 1	120/120	119/119	116/116

and the nine boundary expectations were met. These counts describe the tested fixture, not independent trials or a safety rate. No readback-mismatch stimulus was included, so this experiment does not assess that check.

Separate scripted probes reached the expected approval and authorization branches. In the manual approval fixture, the approved request changed the simulated value from 202 to 204.8, while the rejected request left it at 216. The corresponding automatic calls did not form clean matched pairs because their labels and values did not align with the manual arms. No operator made the approval decisions. The probes show route handling in the tested fixture, not the effect of human approval or a general safety rate.

In a November 5 follow-up, a separate four-scene probe batch returned `RECIPE_LIMIT_EXCEEDED` for all 110 tested requests outside their configured recipe windows but within structural limits. The deterministic inputs and checks derive from the same configuration, so the count assesses the selected cases rather than a population failure rate. In a separate simulated injection-node fault injection, a 230-unit command produced a 251-unit readback against a 0.7-unit tolerance and was reported as `unverified`; one baseline and one recovery returned `readback-verified`. This tests one injected mismatch, not physical-device behavior. The later batch is not pooled with the frozen core results; its records and provenance limits are in Supplementary File S7.

In a separate production-mode check, one legal baseline was accepted and one case each was rejected for interval, step, node-range, recipe-window, and hold-window constraints. A three-case authorization probe rejected an unbound request and a cross-node request and accepted a bound write. These individual branch checks complement the benchmark-mode ablation but do not measure coverage across configurations. Full call records are listed in S7.

C. Process-Target Outcomes

In the no-write control, each simulator ran for 60 s and the endpoint was the mean of its final five samples. The scripted setpoint policy recorded a final process value after scenario-specific steps. The recorded 150.41–172.90 s range is the wall time of each three-scene parallel batch, including setup, rather than a per-scene time to target. Because the horizons and endpoint rules were not matched, the comparison describes these tested routines and does not isolate a control-policy effect. There was no PID or model-predictive-control baseline.

Across the injection, wastewater, and annealing scenes, the scripted policy ended in the target band in 15/15 runs; the no-write control ended in the band in 0/15. The no-write seed-42 injection and wastewater runs began from residual states (32.378 g and 2.894 mg/L, respectively), both already within their target bands; the scripted runs began from blueprint states. The 12 blueprint-matched pairs therefore use seeds 43–46 across the three scenes, with 12/12 scripted endpoints and 0/12 no-write endpoints in band. The 15-run totals include the seed-42 records. We report counts rather than a significance test because the runs share three scenes and the routines have different stopping rules. Fig. 5 shows the seed-level endpoints and the guard-ablation stimuli; neither panel estimates a population-level effect.

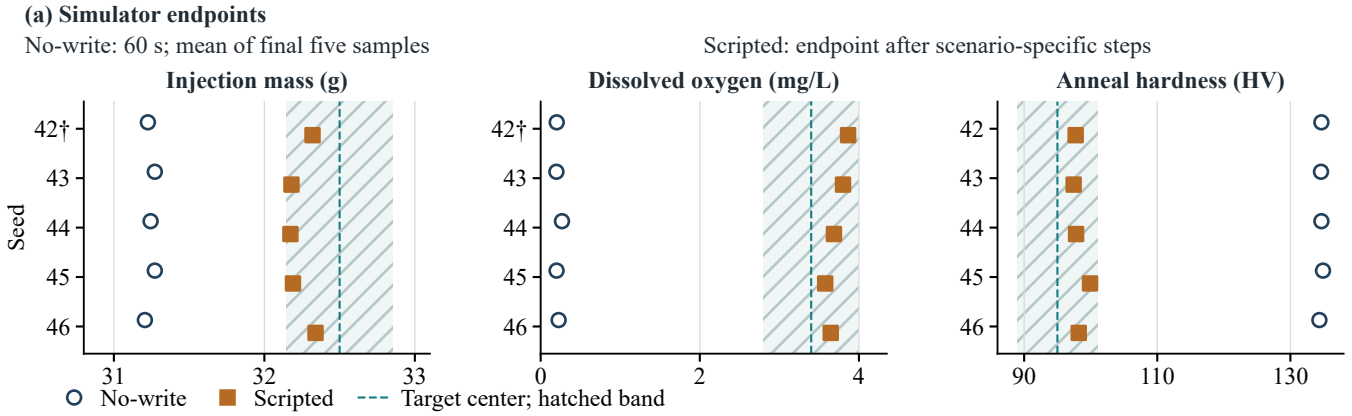
Band membership was recomputed from each recorded process value, target, and tolerance, rather than from the runner's condition-specific `attained` field. The policy used the `recipe_trial` batch route. Across its five seeds it issued 12 writes in 17 rounds for injection, 56 writes in 59 rounds for wastewater, and 41 writes in 50 rounds for annealing. These counts describe the tested missions; they do not compare efficiency with another controller. Seeds are simulator initializations, not independent production lines.

Exact per-seed endpoints and recomputed band membership are listed in Table 3 of Supplementary File S7.

D. Follow-Up Injection-Twin Pilot

An acceptance script later ran one end-to-end experiment on an isolated platform connected to a PLC-based simulated twin; the manifest records `realPlant=false`. At 50 bar, the structured record gives a baseline product weight of 31.503 g, below the 32.00–32.30 g band. Its accompanying report instead gives 31.508 g; the discrepancy remains unresolved. A three-level plan tested 50, 55, and 60 bar. The saved ledger labels its contrasts `no_resolvable_effect`, `underpowered`, and `identifiable_effect`; it stores effect, minimum detectable effect (MDE), and verdict fields. The archive lacks the sample series and the rule used to calculate MDE and assign the labels. The final 55-to-60 bar contrast stores an effect of 0.2517 g over a 5-bar step; 0.05035 g/bar is that single-run ratio, not a replicated or independently verified process model.

The subsequent governed write changed the setpoint from 60 to 63.7 bar, and the device readback matched. After 60 s, the mean of the three latest 15-s process samples was 32.224 g, within the band. The reported 15/15 counts assertions within this one execution, not trials. The record lacks a platform commit, simulator/configuration snapshot, and runner-source hash. This pilot demonstrates one scripted simulator path only; its records and provenance limits are in S7.



(b) Process-window ablation

	Repetition 1		Repetition 2		Repetition 3		
	Below	Above	Below	Above	Below	Above	HTTP 200 / 6
Full guard	○	○	○	○	○	○	0/6
Window check removed	■	■	■	■	■	■	6/6
Readback check removed	○	○	○	○	○	○	0/6
Window + readback removed	■	■	■	■	■	■	6/6

Open circle: HTTP 400 (rejected) | Filled square: HTTP 200
HTTP status does not independently establish a driver call.

Fig. 5. Descriptive simulator endpoints and targeted guard-ablation results. (a) No-write endpoints are final-five-sample means at 60 s; scripted endpoints were recorded after scenario-specific steps. The three-scene scripted batches, including setup, took 150.41–172.90 s; these are not per-scene times to target. The no-write seed-42 injection and wastewater runs began from residual states (†); the matched subset uses seeds 43–46. Hatched regions show target bands and dashed lines their centers. The no-write control was in band in 0/15 runs and the scripted policy in 15/15; the comparison is not time-matched. (b) Each marker represents one below- or above-window request in three technical repetitions. Open circles denote rejection, filled squares denote HTTP 200 responses, and the rightmost fractions count HTTP 200 responses. Structural checks remained enabled.

E. Live-Agent Pilot

Four setup attempts produced two recorded live-Agent episodes, and neither issued a governed write. One episode ended with a `COMPLETED` status while the process value remained outside its target band; the other timed out. The first result shows that orchestration status alone did not verify the process goal. The positive process results above therefore characterize scripted or configured runtime paths, not autonomous-Agent performance. Two additional November 5 episodes were canceled after 371.63 and 372.24 s, respectively; both recorded zero rounds and zero writes. These follow-up episodes do not establish why execution stopped. S7 records the episodes and the earlier setup attempts that produced no result rows.

VII. Discussion

The results demonstrate selected configured operations and process-window checks in simulator and mock-driver fixtures. Their scope is prototype integration, not process-control efficacy or deployment readiness. The target-band comparison is descriptive: the no-write control ran for 60 s and used a fixed endpoint, whereas the scripted policy recorded an endpoint after scenario-specific steps. The scene–protocol matrix is incomplete, the guard ablation used a mock driver without a readback-mismatch stimulus, and approval decisions came from a fixture. The follow-up injection result is one scripted run on a simulated twin; the live-Agent pilot issued no governed writes.

One live-Agent episode was marked `COMPLETED` even though the process variable remained outside its target band. A task status therefore cannot substitute for a separate process observation. The experiments also do not validate every control–measurement pairing. Attribution of a process change to a write requires a commissioned signal relationship, an appropriate observation delay, and an evaluation design capable of supporting that inference.

The November 1 manifests identify platform commit `da627d21`, but the active services were not captured as a hermetic snapshot; available runner inputs and records are retained in Supplementary File S7. Application checks complement PLC, SIS, and equipment protections. Simulator and mock-driver results do not establish physical-plant safety. A process-efficacy study should reset each scene to the same initial state, randomize paired condition order, apply a common fixed horizon, and predefine trajectory metrics such as integrated absolute error, time in band, overshoot, and write count; it should compare against a named rule-based or PID baseline and report scene-stratified uncertainty. Governance coverage should separately test context changes, permission revocation, approval rejection or timeout, and readback mismatch across configurations. Claims about operator approval or live-Agent control require corresponding operator or verified live-harness

trials.

VIII. Conclusion

This work presents a configuration-driven runtime that binds production context and direction-specific signals to registered drivers and governed operations. Its design separates the context supplied to a task from the checks applied at execution, and separates a command result from subsequent process evidence. These distinctions make the applicable route and the meaning of its records explicit; they do not by themselves guarantee atomic execution or establish a causal process effect. The probes and mock-driver ablation support the tested mappings and guard paths; scripted missions provide a descriptive target-band comparison. In a later single-run injection-twin pilot, an acceptance script recorded a governed setpoint update with matching readback and an in-band process observation. The reported gain ratio cannot be independently checked from the archived data. The live-Agent pilot issued no governed writes. The evidence supports selected simulator integration paths, not autonomous-Agent performance, operator-approval effects, or physical-plant safety.

Declarations

Author contributions are declared in the submission system; no competing interests are declared.

Data Availability

Supplementary File S7 contains run manifests, raw CSVs, per-seed outcomes, guard ablations, and scenario-pilot records. The corrected November 5 follow-up archive contains later simulator probes, a readback-mismatch injection, and Agent outcomes. No physical-plant data were collected.

References

[1] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," 2026, arXiv:2610.01364; accepted at IEEE ETFA 2026. [Online]. Available: https://arxiv.org/abs/2610.01364

[3] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[4] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *Journal of Process Control*, vol. 167, p. 103855, 2026. [Online]. Available: https://doi.org/10.1016/j.jprocont.2026.103855

[6] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[7] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[8] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[9] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[10] Z. Li, H. Ge, G. Tang, and Y. Liu, "ModSolAgent: Automated finite element code generation for Abaqus via LLM-based agent," *IEEE Trans. Ind. Informatics*, vol. 22, no. 6, pp. 4693–4704, 2026, doi: 10.1109/TII.2026.3669495.

[11] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[12] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[13] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2026, version 2, Feb. 5, 2026. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[14] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[15] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[16] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[17] M. Xavier, M. Jolly, V. M A, and M. Xavier, "IndustriConnect: MCP adapters and mock-first evaluation for AI-assisted industrial operations," 2026, arXiv:2603.24703 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2603.24703

[18] Y. Liu, X. Xu, and J. Polzer, "Agentic AI-enabled semantic commissioning of a cognitive digital twin for reconfigurable manufacturing," in *Proc. IEEE 22nd Int. Conf. on Automation Science and Engineering (CASE)*, 2026, doi: 10.1109/CASE69030.2026.11704436.

[19] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[20] Y. Zhou *et al.*, "PLCBench: Can autonomous LLM agents turn PLC access into sustained physical impact?" 2026, version 1, submitted Aug. 27, 2026. [Online]. Available: https://arxiv.org/abs/2608.26882

[21] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[22] ISA, "ANSI/ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[23] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[24] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2001.

[25] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.